

vLLM专题（十五）-解耦预填充（Disaggregated Prefilling (experimental)）

 大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续费

 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

本页面向您介绍 vLLM 中的解耦预填充功能。

注意

此功能是实验性的，可能会发生变化。

一、为什么需要解耦预填充？

两个主要原因：

1. 分别优化首 token 延迟（TTFT）和 token 间延迟（ITL）
- 解耦预填充将 LLM 推理的预填充和解码阶段放在不同的 vLLM 实例中。这使您可以灵活地为预填充和解码阶段分配不同的并行策略（例如 `tp` 和 `pp`），从而在不影响 ITL 的情况下优化 TTFT，或者在不影响 TTFT 的情况下优化 ITL。
2. 控制尾部 ITL
- 如果没有解耦预填充，vLLM 可能会在解码一个请求的过程中插入一些预填充任务，这会导致更高的尾部延迟。解耦预填充帮助您解决此问题并控制尾部 ITL。虽然通过适当的块大小进行分块预填充也可以实现相同的目标，但在实践中很难确定正确的块大小值。因此，解耦预填充是控制尾部 ITL 的更可靠方法。

注意

解耦预填充不会提高吞吐量。

二、使用示例

请参阅 [examples/online_serving/disaggregated_prefill.sh](#) 以了解解耦预填充的示例用法。

三、基准测试

有关分解预填充基准测试，请参阅 [benchmarks/disagg_benchmarks/](#)。

四、开发

我们通过运行两个 vLLM 实例来实现分解预填充。一个用于预填充（我们称之为预填充实例），另一个用于解码（我们称之为解码实例），然后使用连接器将预填充实例的 KV 缓存和结果传输到解码实例。

所有分解预填充的实现都位于 [vllm/distributed/kv_transfer](#) 中。

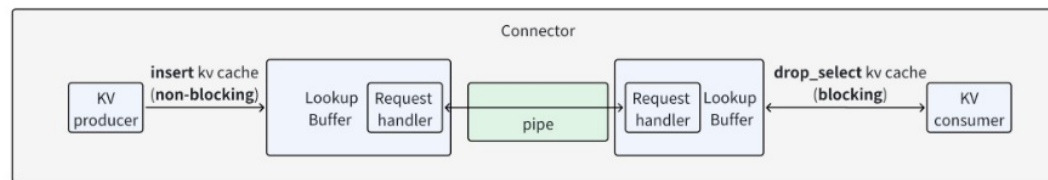
分解预填充的关键抽象：

- **Connector（连接器）**：连接器允许 KV 消费者从 KV 生产者那里检索一批请求的 KV 缓存。
- **LookupBuffer（查找缓冲区）**：LookupBuffer 提供两个 API：插入 KV 缓存和选择删除 KV 缓存。插入和选择删除的语义类似于 SQL，其中插入会将 KV 缓存插入到缓冲区中，而选择删除会返回匹配给定条件的 KV 缓存并将其从缓冲区中删除。
- **Pipe（管道）**：单向 FIFO 管道，用于张量传输。它支持 [send_tensor](#) 和 [recv_tensor](#) 操作。

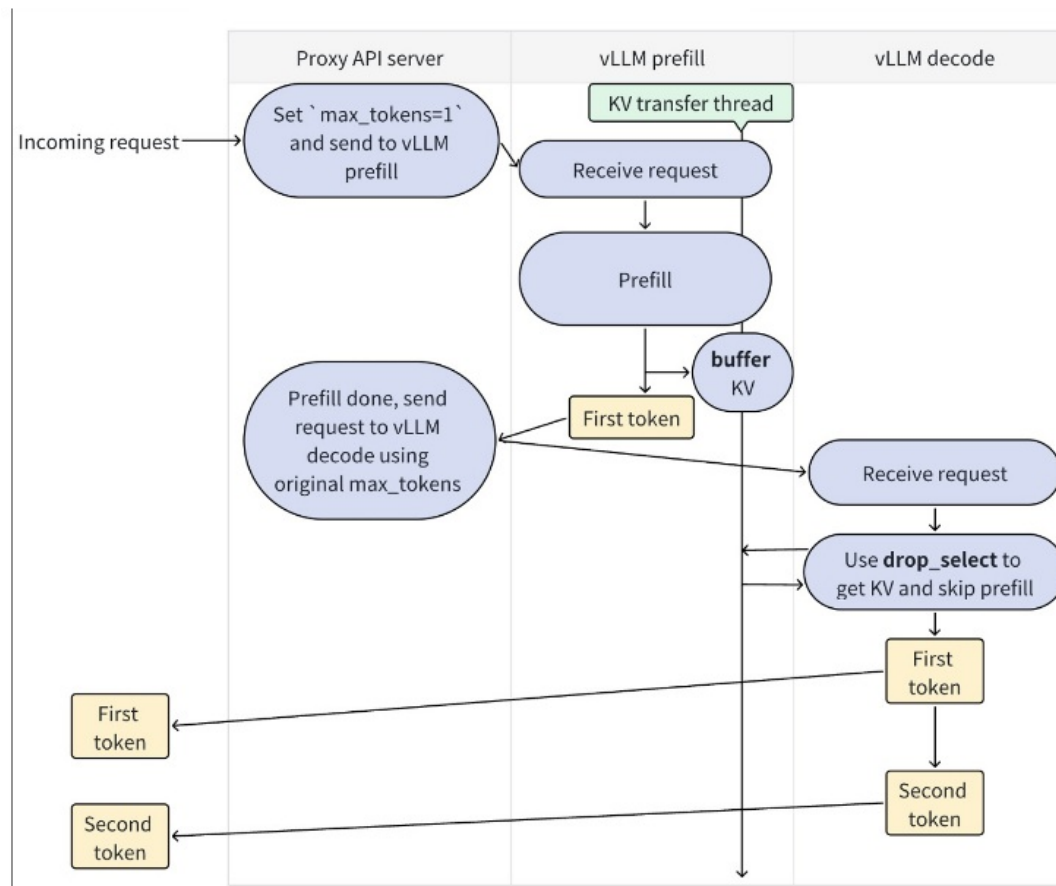
注意：

[insert](#) 是非阻塞操作，但 [drop_select](#) 是阻塞操作。

下面是一个示意图，展示了这三个抽象如何组织：



分解预填充的工作流程如下：



缓冲区对应于 `LookupBuffer` 中的 `insert` API，而 `drop_select` 对应于 `LookupBuffer` 中的 `drop_select` API。

五、第三方贡献

分解预填充与基础设施密切相关，因此 vLLM 依赖第三方连接器来实现生产级别的分解预填充（vLLM 团队会积极审核并合并新的第三方连接器的 PR）。

我们推荐三种实现方式：

完全自定义连接器：实现你自己的连接器，并调用第三方库来发送和接收 KV 缓存，以及其他更多操作（比如编辑 vLLM 的模型输入以执行自定义预填充等）。这种方法提供了最大的控制权，但也存在与未来 vLLM 版本不兼容的风险。

类数据库连接器：实现你自己的 LookupBuffer，并像 SQL 一样支持 `insert` 和 `drop_select` API。

分布式 P2P 连接器：实现你自己的 Pipe，并支持 `send_tensor` 和 `recv_tensor` API，类似于 `torch.distributed`。